

Verifier Bias Is Not Enough: A Controlled Re-examination of Capability Collapse in Iterative Self-Training of Small Reasoning Models

Tan Sang Nguyen¹ and Second Author¹

Ton Duc Thang University, Ho Chi Minh City, Vietnam
52300057@student.tdtu.edu.vn, second.author@example.edu

Abstract. Iterative self-training loops of the form generate $\rightarrow$ verify $\rightarrow$ select $\rightarrow$ train (GVT) improve small language models on mathematical reasoning without human labels, assuming the verifier reliably separates correct from incorrect solutions. We tested whether a rule-based verifier with a *directional* bias — rejecting correct solutions in unexpected formats while almost never accepting incorrect ones — causes capability collapse over several rounds. We measured the bias on `math-verify: \dfrac{\tfrac{1}{2}}{\tfrac{1}{3}}` rewrites of correct answers are rejected 100% of the time. Five GVT rounds on a *pooled* GSM8K + MATH-500 set appeared to confirm the hypothesis: Qwen2.5-1.5B pass@1 fell 50.0% $\rightarrow$ 18.6% and pass@8 72.6% $\rightarrow$ 48.6%. That design, however, trains one shared LoRA adapter on the pooled filtered set and scores on the selection items, separating neither verifier bias from cross-dataset interference nor capability loss from fit to the evaluation set. We re-ran with each dataset isolated on its own adapter, a held-out 30% exam split, and the control the original lacked: the same run with the format bias patched. The collapse disappears — pass@8 is flat throughout — and patching changes nothing measurable across two paired seeds. The reason is prevalence: the affected format occurs in $\approx 0.55\%$ of generations and never on GSM8K, so a 100% rejection rate discards half a percent of the training pool. Published verifier-bias rates are conditional severities; practical impact is severity $\times$ prevalence, which must be measured in the target pipeline. Our striking initial collapse was substantially an artifact of the pooled, non-held-out design.

Keywords: self-training · verifier bias · mathematical reasoning · small language models · experimental confounds · pass@k.

1 Introduction

A common recipe for improving the mathematical reasoning of a small language model (SLM) *without additional human annotation* is to let the model generate its own solutions, keep those that an automated verifier judges correct, and fine-tune the model on the kept solutions, repeating the process for several rounds.

We refer to this recipe as the *Generate–Verify–Train* (GVT) loop. It underlies a family of methods, including STaR [14], ReST [9] and ReST-EM [8], and rejection-sampling fine-tuning (RFT) [12]. Its appeal is that the verifier plays the role of a teacher that needs only to decide right from wrong; no reference solutions are required.

Every method in this family rests on an implicit assumption: the verifier separates correct from incorrect solutions reliably. This assumption is routinely violated when the verifier is *rule-based* — the most common choice for mathematics, because it is cheap, fast, and needs no training of its own. A rule-based verifier such as `math-verify`¹ extracts the final answer from a solution and compares it to the gold answer after normalisation. It does not read the reasoning, and its normalisation step never covers every equivalent way of writing an answer. As a result it produces *false negatives*: it rejects solutions that are correct but written in a form it does not recognise. Crucially, this error is *directional*. Because the verifier compares an extracted final answer against the gold, it almost never accepts a wrong answer — a false positive would require an incorrect answer that normalises exactly to the gold, which is rare. Its errors are therefore overwhelmingly false negatives, all pushing in one direction.

We measured this bias directly on `math-verify` (Section 3.2). On GSM8K, whose answers are simple integers, the false-negative rate on genuinely correct solutions is effectively zero. On MATH-500, whose answers contain fractions and non-trivial L^AT_EX, it rises to roughly 7.8%, and some individually valid rewrites — for instance writing `\dfrac` instead of `\frac` — are rejected 100% of the time even though the numeric value is correct. The bias is one-sided: we did not observe the verifier accepting incorrect final answers.

This paper set out to ask a single question: *when a directionally biased verifier of this kind is used as the filter inside a GVT loop, and the loop is run for many rounds rather than once, what is the consequence — does the model actually improve at reasoning, does it merely learn to write in the style the verifier prefers, or does it lose capability?*

Our first experiments appeared to answer it decisively. Running five rounds on a pooled GSM8K + MATH-500 set, both pass@1 and pass@8 fell sharply for two model sizes, with the larger model collapsing harder. Taken at face value this is a striking result, and an earlier version of this work reported it as evidence that format-biased verifiers cause capability collapse.

That conclusion does not survive a controlled re-examination, and reporting why is the main contribution of this paper. The pooled design has two properties that make it unable to answer the question it was built for. First, it trains *one shared* LoRA adapter each round on the union of both datasets’ accepted solutions, of which GSM8K supplies roughly 69%; what happens on MATH-500 therefore cannot be separated from what happens on GSM8K. Second, it evaluates on the same problems used to select training data, so any round-over-round change mixes genuine capability change with fit to the evaluation items.

¹ <https://github.com/huggingface/math-verify>, version 0.9.0 with antlr4 4.13.2 in this work.

We rebuilt the experiment to remove both properties — one dataset per run, one adapter per dataset, and a held-out 30% exam split never used for selection — and added the single-variable control the original design lacked: an otherwise identical MATH-500 run with the verifier’s format bias patched.

Under the corrected design, the collapse largely disappears and the control comes out null. Our own original hypothesis is not supported by the data that were built to test it properly.

Contributions.

- We characterise the directional bias of a real rule-based verifier (**math-verify**) by format probe and by manual false-negative audit, rather than simulating a noise model.
- We run the GVT loop under two designs — a *pooled* design representative of common practice, and an *isolated* design with per-dataset adapters and a held-out exam split — and show that the two give qualitatively different answers to the same question: dramatic collapse under the first, essentially flat pass@8 under the second.
- We report the single-variable control that the pooled design could not support: repairing the verifier’s format bias, with everything else held fixed, produces no measurable difference across two paired seeds. On this evidence the hypothesis that directional verifier bias drives the decline is *not supported*.
- We identify why, and argue the point generalises: the bias is severe when triggered (100% rejection) but rare in practice (the affected format appears in $\approx 0.55\%$ of generations, and never on GSM8K). Reported verifier-bias rates are conditional severities; their practical impact is severity $\times$ prevalence, and prevalence must be measured in the pipeline where the verifier will actually be used.
- We draw the methodological consequence: GVT-style experiments that pool several datasets into one training mixture, or that score on the selection set, can manufacture a collapse signal that isolation and held-out evaluation do not reproduce.

2 Related Work

Iterative self-training for reasoning. STaR [14] introduced the generate–filter–retrain loop for reasoning without hand-written rationales. ReST [9] and ReST-EM [8] formalise it as alternating “grow” and “improve” steps; ReST-EM applies it to MATH and APPS with PaLM-2, sampling 32 solutions per problem, and reports diminishing test gains after the first round, plus a one-round regression on APPS that it does not investigate. RFT [12] is the single-round rejection-sampling special case. All of these treat the correctness signal as trustworthy; none analyses a verifier that is biased in a fixed direction, and none tracks output style across rounds.

Verifier and reward error. TinyV [11] measures that a rule-based verifier rejects a large fraction of correct solutions and *repairs* the verifier with an auxiliary LLM check, evaluated in a single pass. From Accuracy to Robustness [5] reports that rule-based verifiers for mathematical RL achieve only about 86% recall on average (i.e. $\sim 14\%$ false negatives), an independent figure consistent with ours, but runs RL for a few hundred steps within one training phase and does not track how verifier recall interacts with self-training rounds. Imperfect Verifiers [1] gives the most formal noise model: the verifier flips its decision with fixed probabilities ρ_0 (false accept) and ρ_1 (false reject). Its Section 3.1 explicitly notes that “real-world noise can depend on content, formatting, prompt style”, but then assumes content-independent fixed rates and studies a single RL round. Our study addresses the multi-round version of that gap; our answer, unlike our initial expectation, is that a measured format-dependent bias of this magnitude does not by itself change the multi-round outcome.

Limits of RL and self-training. Yue et al. [13] show that RL raises pass@1 while the base model wins at large pass@ k , i.e. RL sharpens an existing distribution rather than extending capability. Teacher-Free Self-Training [10] runs three sequential self-training rounds on a free-verifier domain and observes a pass@8 vs. pass@64 crossover — the trained model wins at small sampling budget and loses at large budget — concluding that self-training “amplifies but does not compound”. Its verifier is exact string equality, with “zero learned opinion”. Our isolated runs are closer to this picture than to a collapse picture: with the confounds removed, pass@8 is essentially preserved and what erodes, mildly, is first-attempt reliability.

Model collapse. Training repeatedly on model-generated data erodes distribution tails and converges output toward the mean [7]. Qiao et al. [6] argue theoretically that a selector able to “see” only a narrow slice of the target distribution becomes itself a source of bias that accelerates collapse. Our pooled runs looked like an empirical instance of that mechanism; our isolated runs do not reproduce it, which we read as a caution against inferring the mechanism from aggregate pass-rate curves alone (Section 5).

Single-round verifier-error studies. Egashira et al. [3] classify verifier errors in one-round RLVR on synthetic arithmetic and conclude that systematic false negatives behave like random noise — they slow learning but do not cause collapse — while false positives cause plateau or collapse. An earlier version of this work treated their conclusion as being in tension with ours. With the controlled runs in hand the tension is gone: our single-variable verifier control (Section 4.5) reaches the same conclusion they do, now in a multi-round supervised setting rather than a single RLVR phase, and with a real rather than simulated verifier.

Summary. No prior work combines a real, measured, format-biased rule-based verifier with multiple sequential self-training rounds and a matched patched-verifier control. We close that gap, and the answer is negative: within our measurement precision, the bias does not change the multi-round outcome.

3 Setup

3.1 The GVT Loop

Each round t of our loop performs four steps.

Generate. For every problem in the working set, the current model samples $k = 8$ solutions (temperature sampling).

Verify. `math-verify` (version 0.9.0, antlr4 4.13.2) extracts the final answer from each solution and compares it to the gold answer.

Select. Solutions the verifier accepts are kept; all others are discarded.

Train. A *fresh* LoRA adapter (rank 16, $\alpha = 32$) is attached to the *base* model and trained by supervised fine-tuning on the kept solutions for one epoch. Adapters are *not* accumulated across rounds: round t starts from the base weights and learns only from round $t - 1$'s accepted pool.

The loop is run for five rounds ($t = 0, \dots, 4$), where $t = 0$ is the untrained base model.

3.2 The Verifier and Its Directional Bias

Before running the loop we characterised `math-verify` in two ways.

Format probe. We took gold answers from GSM8K and MATH-500 and re-rendered each in a set of individually valid formats (bare number, trailing-zero decimal, `\frac`, `\dfrac`, `\tfrac`, boxed, dollar-delimited, full sentence, with units), then asked the verifier to grade the re-rendered answer against the original gold. On GSM8K, plain rewrites are never rejected; `\dfrac` and `\tfrac` are rejected 200/200 (the verifier fails to parse the command). On MATH-500, `\dfrac`/`\tfrac` are rejected 146/146 (100%), trailing-zero decimals such as 0.50 are rejected 59% of the time, and boxed/dollar-delimited forms are rejected about 20% of the time.

False-negative rate on real solutions. We then took genuinely correct model-generated solutions (public teacher traces for GSM8K; public Qwen2.5-7B roll-outs for MATH-500) and manually checked, for each solution the verifier rejected, whether the boxed answer actually matched the gold. The false-negative rate among correct solutions is **7.8%** on MATH-500 (10/128) and **0%** on GSM8K (0/200); the two verifier presets we tested (`default` and `reward`) give identical counts. Of the ten MATH-500 false negatives, eight are cases where the verifier extracts the wrong token from a long solution, and two are valid rewrites it cannot match (`\left( 3, \frac{\pi}{2} \right)` vs. `(3, \frac{\pi}{2})`; `6+9i` vs. `6 + 9i`). We did not observe the verifier accepting an incorrect final answer; the error is one-directional.

The patch. For the control experiment we repaired exactly the largest identified failure mode: a normalisation pass that rewrites `\dfrac` and `\tfrac` to `\frac` before the verifier's own parsing. This removes the 100%-rejection format class from the verifier's error profile while leaving every other behaviour untouched.

Table 1. The two designs. Design I removes the pooling confound and the selection/evaluation overlap, and adds the patched-verifier control.

	Design P (pooled)	Design I (isolated)
Datasets per run	GSM8K + MATH-500	one only
Adapter	one, shared	one per dataset
Training mixture	$\approx 69\%$ GSM8K	single-domain
Evaluation items	same as selection	held-out 30%
Verifier control	none	Run B (patched)
Models	0.5B and 1.5B	1.5B
Runs	2	3 (A, B, C)

3.3 Two Experimental Designs

Our results come from two designs, summarised in Table 1. The distinction is the central point of this paper, so we state it explicitly rather than as a configuration detail.

Design P (pooled). Both datasets are loaded together, and each round trains *one* LoRA adapter on the union of the accepted solutions from both. Selection and evaluation use the same problems. This is the design of our initial experiments and, in structure, of much reported practice.

Design I (isolated). One dataset per run, one adapter per run, and a held-out split: 30% of the problems (150 of 500) form an `exam` set that is *never* used to select training data, while the remaining 350 form the `select` set. All reported pass rates under Design I are computed on the held-out exam set only.

Design I comprises three runs, all Qwen2.5-1.5B-Instruct, $n = 500$, five rounds, $k = 8$, `holdout-frac` = 0.3, seed 0, differing in exactly one respect each:

Run A. MATH-500 only, original verifier. The dataset where the bias is present, isolated.

Run B. MATH-500 only, *patched* verifier. Identical to Run A in every other respect; the single-variable control for the paper’s central hypothesis.

Run C. GSM8K only, original verifier. The dataset where the verifier is already near-clean (0% measured false negatives).

Run A was additionally repeated with seed 1. Because the seed governs the train/exam partition as well as sampling, the repetition draws a *different* 150-problem exam set, which lets us calibrate how much of any observed movement is attributable to the choice of evaluation items (Section 4.4).

3.4 Models, Data, Training, Metrics

Models. Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct.² Design P uses both; Design I uses the 1.5B model only.

² <https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>, <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>.

Table 2. Design P: pass@1 and pass@8 over five GVT rounds, pooled GSM8K + MATH-500, scored on the selection set. Round 0 is the untrained base model. Section 4.2 explains why these numbers cannot be attributed to verifier bias.

Model	Metric	R0	R1	R2	R3	R4	Δ (R0→R4)
0.5B	pass@1	31.1%	31.9%	29.9%	28.9%	27.4%	−3.7 pt
0.5B	pass@8	60.5%	58.9%	56.1%	55.5%	54.9%	−5.6 pt
1.5B	pass@1	50.0%	46.2%	44.5%	32.4%	18.6%	− 31.4 pt
1.5B	pass@8	72.6%	71.5%	70.2%	63.6%	48.6%	− 24.0 pt

Data. GSM8K [2] and MATH-500 — the 500-problem subset of MATH [4] — 500 problems each, drawn from the test splits.

Training. LoRA supervised fine-tuning (`transformers` + `peft` + TRL `SFTTrainer`), one epoch per round at a learning rate of 10^{-5} in every run reported here. We use SFT on verifier-accepted solutions rather than online RL because it matches the select-then-train structure of the loop.

Metrics, per round. (i) pass@1 and pass@8; (ii) style counts over all generations in the round, for labels including `boxed`, `dfrac`, `frac`, `dollar`, `sentence`, `assignment`, `decimal`; (iii) pre-filter answer diversity: the mean fraction of distinct normalised final answers among the k samples for a problem, and the fraction of problems on which all k samples agree; (iv) for Design I, the final training loss reported by `SFTTrainer` on that round’s kept pool.

Infrastructure. A single Kaggle T4 GPU. Design P and Run C were run once; Runs A and B were run at two seeds (Section 4.4 and Section 6).

4 Results

4.1 Design P: An Apparent Collapse

Table 2 reports pass@1 and pass@8 for both models across the five rounds of the pooled design; the two verifier presets produced identical values, so a single column is shown. All four series decline monotonically or nearly so (the only exception is a +0.8-point blip in 0.5B pass@1 at round 1), and the 1.5B decline is severe and accelerating.

Splitting the 1.5B pooled run by dataset (Table 3) sharpens the puzzle rather than resolving it. Under the original hypothesis, MATH-500 — where the verifier’s false-negative rate is 7.8% — should degrade and GSM8K — where it is 0% — should hold steady. Instead both fall heavily, and GSM8K, the dataset with the clean verifier, loses more pass@1 in absolute points than MATH-500.

The accepted training pool also contracts every round — mildly for the 0.5B model (−16% by round 4) and sharply for the 1.5B model (−60%, from 3,962 to 1,566 solutions) — and 1.5B output style shifts markedly, with `\boxed{}` occurrences falling from 5,608 to 2,693 while dollar-delimited math rises from 1,453 to 3,406. Read on its own, this is a coherent story about a shrinking, style-skewed pool driving capability down.

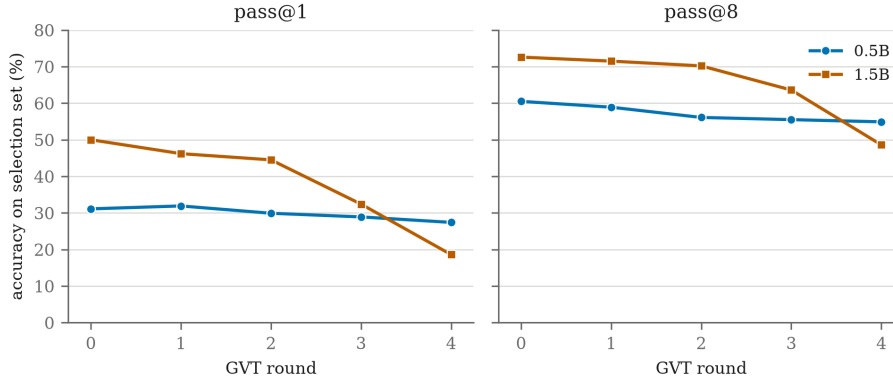


Fig. 1. Design P: pass@1 and pass@8 across five rounds for both model sizes. Both metrics fall for both models. Section 4.3 shows that this pattern does not survive isolation and held-out evaluation.

Table 3. Design P, 1.5B, split by dataset. GSM8K has a measured 0% verifier false-negative rate yet declines at least as much as MATH-500.

Dataset	Metric	R0	R1	R2	R3	R4	Δ
GSM8K	pass@1	69.6%	62.0%	61.0%	45.6%	26.6%	−43.0 pt
GSM8K	pass@8	91.4%	90.4%	89.0%	82.2%	65.8%	−25.6 pt
MATH-500	pass@1	30.4%	30.4%	28.0%	19.2%	10.6%	−19.8 pt
MATH-500	pass@8	53.8%	52.6%	51.4%	45.0%	31.4%	−22.4 pt

4.2 Why Design P Cannot Answer the Question

Two properties of Design P block the inference.

Shared adapter over a pooled mixture. Each round trains one adapter on the union of both datasets’ accepted solutions. Table 4 shows the composition of that union: GSM8K supplies between 68.4% and 70.2% of it in every round. The adapter evaluated on MATH-500 is therefore one that learned roughly 69% of its update signal from GSM8K, and vice versa. Whatever happens to either dataset’s score is a joint function of both datasets’ filtered pools, and no contrast within this design isolates the verifier’s contribution. We note that the mixture ratio is *stable* across rounds, which rules out a drifting-ratio explanation but does not remove the interference itself.

Selection and evaluation on the same problems. Design P scores each round on the problems it also used to choose training data. A round-over-round change therefore mixes capability change with fit to those specific items, and neither component can be recovered from the aggregate.

Table 4. Design P, 1.5B: composition of the shared training pool. GSM8K dominates the mixture in every round, so neither dataset’s trajectory is independent of the other.

Round	0	1	2	3	4
Kept, GSM8K	2725	2551	2356	1837	1100
Kept, MATH-500	1237	1179	1039	800	466
GSM8K share	68.8%	68.4%	69.4%	69.7%	70.2%

Table 5. Design I: pass@1 and pass@8 on the held-out exam set (150 problems), Qwen2.5-1.5B. Runs A and B differ only in whether the verifier’s format bias is patched. Run A is shown for both seeds; seed 1 draws a different exam partition, so its round-0 value differs (Section 4.4).

Run	Metric	R0	R1	R2	R3	R4	Δ
A, seed 0 (MATH-500, orig.)	pass@1	32.0%	29.3%	28.0%	29.3%	29.3%	−2.7 pt
A, seed 0 (MATH-500, orig.)	pass@8	57.3%	56.7%	54.7%	57.3%	56.0%	−1.3 pt
A, seed 1 (MATH-500, orig.)	pass@1	36.7%	32.7%	32.7%	31.3%	33.3%	−3.4 pt
A, seed 1 (MATH-500, orig.)	pass@8	58.7%	60.7%	59.3%	59.3%	58.7%	0.0 pt
B, seed 0 (MATH-500, patched)	pass@1	32.0%	28.7%	28.7%	28.7%	28.7%	−3.3 pt
B, seed 0 (MATH-500, patched)	pass@8	57.3%	55.3%	52.0%	57.3%	53.3%	−4.0 pt
B, seed 1 (MATH-500, patched)	pass@1	37.3%	32.0%	32.7%	31.3%	33.3%	−4.0 pt
B, seed 1 (MATH-500, patched)	pass@8	59.3%	60.7%	58.0%	58.7%	58.7%	−0.6 pt
C, seed 0 (GSM8K, orig.)	pass@1	66.7%	66.7%	65.3%	63.3%	54.0%	−12.7 pt
C, seed 0 (GSM8K, orig.)	pass@8	92.7%	90.7%	93.3%	90.0%	90.0%	−2.7 pt

4.3 Design I: The Collapse Does Not Reproduce

Table 5 reports the isolated runs on their held-out exam sets. The severe collapse of Table 2 is absent. pass@8 is flat in every run: 57.3% $\rightarrow$ 56.0% and 58.7% $\rightarrow$ 58.7% for Run A at seeds 0 and 1, 57.3% $\rightarrow$ 53.3% for Run B, and 92.7% $\rightarrow$ 90.0% for Run C, against declines of 22–26 points in the pooled design. On MATH-500 — the dataset the hypothesis is about — pass@1 falls by about three points at round 1 and is flat thereafter, and the two seeds of Run A agree on this shape (−2.7 and −3.4 points end to end).

The shape itself is informative, and having two seeds is what makes it readable. In both, essentially the entire pass@1 movement occurs at round 1 (32.0 $\rightarrow$ 29.3 at seed 0; 36.7 $\rightarrow$ 32.7 at seed 1), after which rounds 1–4 show no trend in either direction (29.3, 28.0, 29.3, 29.3 and 32.7, 32.7, 31.3, 33.3). This is a one-off cost of the first self-training round rather than a compounding process — which is the signature the collapse hypothesis predicts and does not find here. It matches the “amplifies but does not compound” reading of [10] more closely than any collapse account.

Figure 3 plots the same data. Run C retains a residual effect worth naming: GSM8K pass@1 falls 12.7 points while its pass@8 does not move. The model does not lose the ability to reach a correct answer within eight attempts; it becomes less reliable at producing one on the first. Almost the entire drop occurs in the

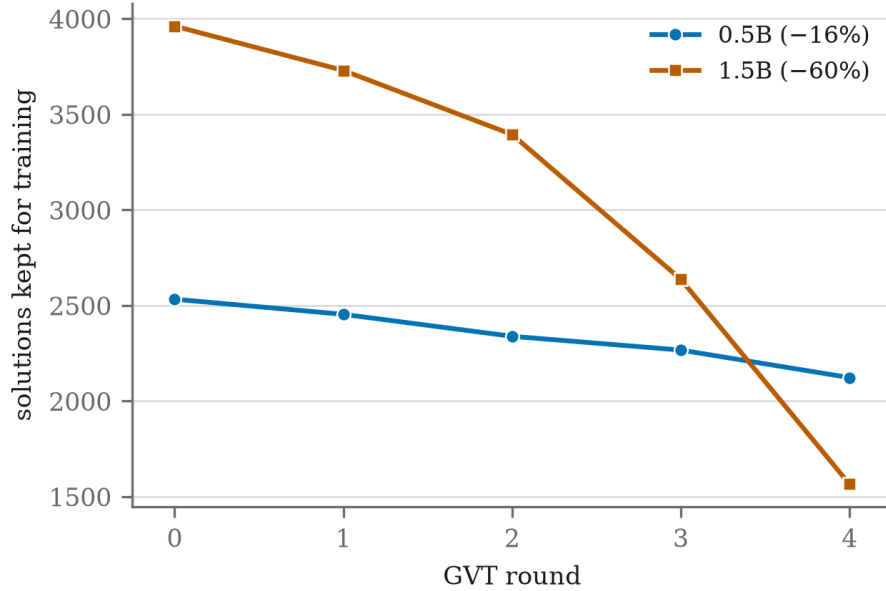


Fig. 2. Design P: size of the verifier-accepted training pool by round. The pool contracts in both runs. Section 4.3 shows pool contraction also occurs *without* a corresponding pass@8 decline, which weakens the shrinking-pool explanation.

final round (63.3% $\rightarrow$ 54.0%), and with a 150-problem exam set and a single seed we cannot presently separate this from noise. We flag it as the one open empirical question these runs leave behind, not as a finding.

The accepted pools under Design I contract as well — Run C from 1,901 to 1,515 kept solutions (−20%), Run A from 872 to 843 (−3%) — yet pass@8 in both runs is flat. Pool contraction and capability collapse, which moved together in Design P, come apart here.

4.4 Calibrating the Noise Floor

The two seeds of Run A also give us something the study previously lacked: a measured scale for how large a difference must be before it means anything. Round 0 is the *untrained base model*, identical in both seeds; the only thing that differs is which 150 problems the seed assigned to the exam partition. Yet round 0 pass@1 is 32.0% at seed 0 and 36.7% at seed 1 — a gap of 4.7 points, or seven problems — with pass@8 differing by 1.4 points.

The figure is worth stating precisely, because it is easy to misuse. It is a *between-partition* variance, and most of the comparisons this paper relies on are *within-partition* and therefore paired: round 0 versus round 4 inside one run scores the same 150 problems, and Runs A and B at seed 0 share a partition. In

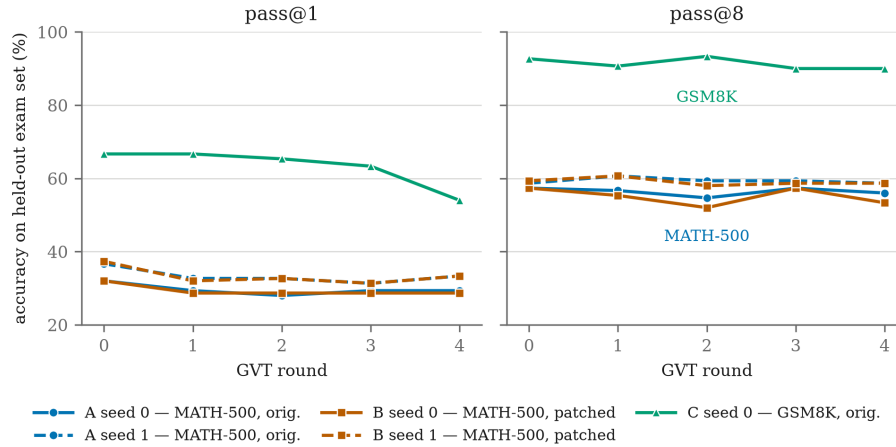


Fig. 3. Design I: pass@1 (left) and pass@8 (right) on the held-out exam set, by round. Colour identifies the run, line style the seed, marker the run again, so the figure survives greyscale printing. Runs A and B — identical except for the verifier patch — track each other closely at both seeds. pass@8 is flat in every run, in contrast to the 22–26 point declines of Design P (Fig. 1). The one visible movement is Run C’s pass@1 in the final round.

those paired comparisons the partition variance largely cancels, so 4.7 points is not an error bar for them. It does bound a different and equally important thing: any comparison of *absolute* levels across runs with different seeds — for instance Run A’s 32.0% at seed 0 against 36.7% at seed 1 — carries several points of partition noise and should not be read as an effect.

With that stated, the number still does useful work. It shows that a 150-problem exam set resolves differences of a few points only weakly, which is the regime the A-versus-B gap of 0.6 points sits in; and it puts the Run C decline of 12.7 points comfortably outside the range that partition choice alone can explain.

4.5 The Control: Patching the Verifier Changes Nothing

Runs A and B are identical in dataset, seed, split, model, and training configuration, and differ only in whether `\dfrac`/`\tfrac` are normalised before verification — the one error class we measured at 100% rejection. This is the single-variable control the paper’s central hypothesis requires, and which Design P could not provide.

The two curves are indistinguishable at our measurement precision (Table 5). Over five rounds pass@1 ends at 29.3% unpatched versus 28.7% patched — a difference of 0.6 points, i.e. one problem out of 150 — and pass@8 at 56.0% versus 53.3%, with Run B’s pass@8 oscillating non-monotonically (55.3 → 52.0 → 57.3 → 53.3) in a manner consistent with sampling noise rather than a trend.

Table 6. Occurrences of the patched format class among the 4,000 generations produced per round. The bias is severe when triggered (100% rejection, Section 3.2) but is triggered rarely.

Dataset	Token	R0	R1	R2	R3	R4
MATH-500	<code>\dfrac</code>	23–29	23	22	21–22	20–22
MATH-500	<code>\tfrac</code>	0	0	0	0	0
GSM8K	<code>\dfrac</code>	0	0	0	0	0
GSM8K	<code>\tfrac</code>	0	0	0	0	0

Pre-filter answer diversity is likewise near-identical between the two (Table 7), as are their per-round training losses ($0.38 \rightarrow 0.43$ for A, $0.39 \rightarrow 0.44$ for B).

The pattern repeats at seed 1, where the agreement is closer still: from round 2 onward the two runs report *identical* pass@1 (32.7%, 31.3%, 33.3%), and their end-to-end changes are -3.4 versus -4.0 points. Across both seeds, no round shows a difference larger than 2.7 points in pass@1, and the differences have no consistent sign.

Repairing the verifier’s measured format bias produces no measurable improvement. Within the precision of this experiment — now two paired seeds — the hypothesis that directional format bias in the verifier drives multi-round degradation is *not supported*. Section 4.6 explains why.

4.6 Why the Patch Does Nothing: Severity Without Prevalence

A null result is more useful when the mechanism behind it is identified, and our style counts supply one directly.

Round 0 gives an unusually clean measurement. At a fixed seed, Runs A and B generate from the same base model with the same sampling seed and have not yet trained on anything, so their round-0 generations are identical — the style counts confirm this exactly (e.g. 29 `\dfrac` occurrences in both runs at seed 1). The *only* difference at round 0 is which verifier scores those identical solutions. The patch’s direct scoring effect is therefore isolated with no confounds at all, and it is: zero problems out of 150 at seed 0 (32.0% both), and one problem out of 150 at seed 1 (36.7% versus 37.3%).

The reason is prevalence. Table 6 counts how often the patched format class actually occurs in the model’s output. Across 4,000 generations per round on MATH-500, `\dfrac` appears in 20–29 of them — about 0.55% — and `\tfrac` never appears at all. On GSM8K, whose answers are integers, `\dfrac` occurs zero times in every round, which is why no patched GSM8K run is worth spending compute on.

This resolves an apparent tension in our own measurements. The format probe of Section 3.2 reports 100% rejection for `\dfrac`/`\tfrac`, which sounds severe, but it measures severity *conditional on the format occurring*. The quantity that governs the training pool is severity $\times$ prevalence: 100% of 0.55% is 0.55% of solutions wrongly discarded per round. A pool reduced by half a percent

Table 7. Pre-filter answer diversity across $k=8$ samples per problem, by round. A lower “% all- k agree” means the model is *less* consistent about what answer it gives to the same problem.

Run / metric	R0	R1	R2	R3	R4
<i>Design P</i>					
0.5B, unique answers / k	0.600	0.601	0.608	0.620	0.649
0.5B, % all- k agree	10.5%	10.8%	9.2%	9.1%	7.3%
1.5B, unique answers / k	0.443	0.482	0.543	0.634	0.712
1.5B, % all- k agree	24.8%	21.4%	14.5%	6.6%	1.7%
<i>Design I</i>					
A (seed 0), unique answers / k	0.542	0.545	0.545	0.546	0.554
A (seed 0), % all- k agree	14.6%	13.4%	13.2%	12.2%	12.0%
A (seed 1), unique answers / k	0.528	0.539	0.548	0.543	0.549
A (seed 1), % all- k agree	16.0%	15.2%	13.2%	13.8%	12.6%
B (seed 0), unique answers / k	0.542	0.546	0.545	0.547	0.555
B (seed 0), % all- k agree	14.6%	13.8%	14.2%	12.2%	12.2%
B (seed 1), unique answers / k	0.528	0.542	0.544	0.545	0.543
B (seed 1), % all- k agree	16.0%	15.2%	14.0%	13.2%	12.8%
C, unique answers / k	0.359	0.364	0.413	0.444	0.500
C, % all- k agree	33.6%	32.4%	25.2%	18.8%	16.0%

cannot plausibly drive a twenty-point capability change. Our own false-negative audit points the same way in retrospect: of the ten MATH-500 false negatives we found, only two were format rewrites, while eight were wrong-token extraction from long solutions — a failure mode the patch does not address.

We think this is the most transferable lesson of the study. Verifier-bias figures in the literature, including the $\sim 14\%$ false-negative rate of [5] and the large rejected fractions of [11], are conditional severity figures measured on some reference distribution. Whether such a bias matters in a given pipeline depends on how often the affected forms appear in *that pipeline’s* model output, which is a cheap thing to count and which we did not count until after the hypothesis had failed.

4.7 Pre-Filter Answer Diversity Increases, It Does Not Collapse

A natural companion question is whether the model converges onto a single *answer* per problem — whether the k raw samples for one problem, *before* the verifier sees them, become more alike as training proceeds. We measured this in both designs by extracting each sample’s final answer and computing, per round, the mean fraction of distinct normalised answers among the $k = 8$ samples and the fraction of problems on which all k agree.

In both designs, answer-level diversity *rises* — the opposite of a “the model narrows onto one preferred output” picture. (Raw-text uniqueness among the k samples stays at ≈ 1.0 every round: at temperature 0.8, independent samples

are almost never byte-identical, so that metric is uninformative here.) The magnitude tracks the magnitude of the pass@1 decline: steep in Design P’s 1.5B run (24.8% $\rightarrow$ 1.7% all- k agreement), moderate in Run C (33.6% $\rightarrow$ 16.0%), and nearly flat in Runs A and B, which are also the runs whose pass@1 barely moves. This is consistent with a single underlying quantity — first-attempt reliability — rather than with stylistic convergence: a model that still solves a problem tends to land on the same answer across attempts, while one whose reliability is eroding answers more erratically, which depresses pass@1 while leaving pass@8 comparatively intact.

5 Discussion

5.1 Withdrawing the Shrinking-Pool Explanation

An earlier version of this work explained the Design P curves with a self-reinforcing shrinking-pool mechanism: the biased verifier accepts a smaller and more style-skewed set each round, the next round trains only on that set, and the skew compounds. The mechanism is coherent, and matches Qiao et al. [6] theoretically. The isolated runs do not support it.

Two observations undercut it directly. First, the pool contracts under Design I as well — 20% in Run C — with no corresponding pass@8 decline, so contraction of this magnitude is not sufficient for capability loss. Second, and more decisively, removing the bias that supposedly drives the skew changes nothing (Section 4.5). A mechanism whose causal ingredient can be removed with no effect on the outcome is not the operative mechanism.

What remains is a narrower and better-supported statement: in the pooled design, each dataset’s adapter update was dominated by the other dataset’s filtered data, and evaluation ran on the selection items. Those two properties are sufficient to produce the observed curves without any appeal to verifier bias, and when both are removed the curves largely flatten.

5.2 What Exactly Differs Between the Designs

Honesty about attribution requires naming a limitation of our own comparison. Design I differs from Design P in three respects at once: datasets are isolated; evaluation moved to a held-out split; and n per run fell from 1,000 pooled problems to 500. We therefore cannot apportion the difference between Table 2 and Table 5 across these three changes.

The held-out split alone plausibly accounts for a substantial share: Design P’s curves rise or hold in early rounds and fall later, which is the shape expected when a model first fits and then departs from the very items being scored. We make no attempt to rank the three factors, and we do not claim that isolation *alone* removes the collapse.

The comparison that *is* clean is internal to Design I: Runs A, B and C share every configuration except the single variable each is meant to vary. The refutation in Section 4.5 rests only on that internal comparison, not on the Design P/Design I contrast.

5.3 Reconciling with One-Round Studies

Egashira et al. [3] report that systematic false negatives in one-round RLVR act like random noise and do not cause collapse. Our controlled result agrees with theirs, and extends it in two directions they did not cover: the multi-round supervised self-training regime, and a real rather than simulated verifier whose bias was measured before use. That two quite different setups converge on “systematic false negatives are tolerable” is, we think, the most useful positive statement this study can make.

5.4 Practical Implications

For practitioners building GVT-style pipelines, our experience suggests three concrete precautions, in descending order of importance.

Do not pool datasets into one adapter if you intend to report per-dataset trajectories. A shared adapter trained on a mixture that is 69% one domain does not yield per-domain results that can be interpreted independently. Isolation costs less than it appears: each isolated run in our study used half the problems of a pooled run.

Hold out evaluation items from selection. Scoring on the selection set is the single change most likely to manufacture a dramatic round-over-round curve.

Include the null control. Our central claim survived two model sizes, tens of thousands of generations and a coherent mechanism story, and failed the first time a single-variable control was run against it. If a pipeline attributes an effect to a specific verifier defect, the cheapest decisive experiment is to repair that defect and re-run.

Finally, the earlier practical advice stands on its own and is unaffected by the reversal: a rising pass@1 across rounds is not sufficient evidence of improvement, and pass@ k should be tracked in parallel. Our Run C illustrates the converse case as well — a falling pass@1 with flat pass@8, which would be misread as capability loss if pass@1 were the only metric reported.

6 Limitations

1. *Limited seed replication.* Runs A and B have each been executed at two seeds, which agree, so the null control rests on two paired comparisons rather than one. Run C and the pooled runs were executed once; replicating Run C is the immediate next step, since its 12.7-point pass@1 decline is the one effect in this study that exceeds the noise floor.
2. *Small exam set.* Design I scores on 150 held-out problems, where one problem is 0.67 points, and re-partitioning alone moves round-0 pass@1 by 4.7 points (Section 4.4). Differences of a few points — including the entire A-versus-B gap — are below the resolution of this setup.
3. *Three variables change between designs* (Section 5.2); the Design P/Design I contrast is therefore suggestive rather than attributive.

4. *Design I covers one model size.* The pooled runs used 0.5B and 1.5B; the isolated runs use 1.5B only, so we cannot say whether the apparent “larger collapses harder” pattern of Design P also dissolves under isolation.
5. *One patch, one bias class.* Run B repairs $\backslash\mathrm{dfrac}/\backslash\mathrm{tfrac}$ parsing, the error class we measured at 100% rejection. It does not repair the eight-of-ten false negatives caused by wrong-token extraction from long solutions, so our null result concerns format-class bias specifically, not verifier error in general.
6. *Five rounds, one verifier, one training method.* Whether the curves change beyond five rounds, under a different verifier, or under full fine-tuning or online RL, is untested.

7 Conclusion

We asked whether a directionally biased rule-based verifier — one that rejects correct-but-differently-formatted solutions and almost never accepts incorrect ones — causes capability collapse when used as the filter in a multi-round GVT self-training loop. Our first experiments, run on a pooled two-dataset set and scored on the selection items, produced a dramatic collapse in both pass@1 and pass@8 that appeared to answer the question affirmatively.

Rebuilding the experiment to isolate each dataset on its own adapter, to score on a held-out split, and to include the matched patched-verifier control changes the answer. The collapse does not reproduce: pass@8 is flat in every isolated run, and the MATH-500 result replicates across two seeds. The control comes out null at both seeds: repairing the verifier’s measured 100%-rejection format class alters neither pass@1, pass@8, answer diversity, nor training loss. The reason is that the affected format occurs in about half a percent of generations, so a rejection rate of 100% conditional on that format removes half a percent of the training pool — severity and prevalence are different quantities, and only the second one governs the pipeline. What remains is a mild first-attempt erosion on GSM8K, larger than the noise floor but so far measured at one seed.

The finding we can defend is therefore methodological rather than mechanistic. Pooling several datasets into one training mixture, and scoring on the problems used for selection, are together sufficient to produce a collapse signature in a GVT loop that isolation and held-out evaluation do not reproduce — and a mechanism story built on that signature survived considerable scrutiny before a single-variable control dismissed it. We report the reversal in full because the pooled design that produced it is not unusual, and because the corrected design was cheaper to run than the one it replaced.

Future work. Repeat Runs B and C at further seeds — Run A is already replicated — to establish whether the Run C pass@1 erosion is real and to bound the size of any effect the verifier patch might have; run the no-filter ablation, which trains on all k samples without verifier selection and thereby separates selection-driven contraction from ordinary self-training on model output; and extend the isolated design to a second model size to test whether the size-dependent severity seen under pooling survives isolation.

A second, independent format-class bug gives a natural next operating point for the same control design. `math-verify`'s rich LaTeX extraction (square roots, complex numbers, tuples, polynomials) only activates inside a recognised delimiter (`\boxed{...}` or `$...$`); undelimited text falls back to a bare-number heuristic that silently drops the rest of the expression. Our gold answers are stored undelimited, so whenever a model correctly boxes a compound answer — exactly what our GVT prompt asks for — gold is parsed as a truncated stub while the prediction is parsed in full, and the two compare unequal. Re-examining the format probe (Section 3.2) attributes only 17/41 of the MATH-500 `boxed` rejections to this mechanism specifically (8.5%; the remaining 24/41 are the pre-existing unparseable gold formats noted above, which fail identically whether or not the answer is boxed), giving an estimated damage of $8.5\% \times 51.5\% \approx 4.4\%$ under the prevalence measured in Section 4.6 — roughly eight times `\dfrac`'s 0.55% and, unlike `\dfrac`, untested by this or any study we are aware of. We implemented and unit-tested the matching patch (wrap gold in `\boxed{}`) whenever the prediction already contains one and gold does not, verified against every writing-style variant from the format probe of Section 3.2 with no regression elsewhere); running it as a Run-D control against the chosen headline configuration, exactly as Run B isolated the `\dfrac` patch against Run A, is queued next.

Acknowledgments. This work was carried out as an undergraduate research project. No external funding was received.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Cai, X.-Q., Wang, W., Liu, F., Liu, T., Niu, G., Sugiyama, M.: Reinforcement Learning with Verifiable yet Noisy Rewards under Imperfect Verifiers. arXiv:2510.00915 (2025)
2. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J.: Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 (2021)
3. Egashira, K., Vero, M., Dekoninck, J., Dörner, F.E., Staab, R., Vechev, M.: Delay, Plateau, or Collapse: Evaluating the Impact of Systematic Verification Error on RLVR. arXiv:2605.02909 (2026)
4. Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J.: Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874 (2021)
5. Huang, Y., Zeng, W., Zeng, X., Zhu, Q., He, J.: From Accuracy to Robustness: A Study of Rule- and Model-based Verifiers in Mathematical Reasoning. arXiv:2505.22203 (2025)
6. Qiao, X., Du, X., Liu, W., Zhang, J., Mai, P., Zhang, M., Pang, Y.: When Sample Selection Bias Precipitates Model Collapse. arXiv:2606.13732 (2026)
7. Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., Gal, Y.: AI models collapse when trained on recursively generated data. Nature (2024). <https://doi.org/10.1038/s41586-024-07566-y>

8. Singh, A., Co-Reyes, J.D., Agarwal, R., et al.: Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models. arXiv:2312.06585 (2023)
9. Gulcehre, C., Le Paine, T., Srinivasan, S., et al.: Reinforced Self-Training (ReST) for Language Modeling. arXiv:2308.08998 (2023)
10. Strozzi, I.L.: Teacher-Free Self-Training Amplifies but Does Not Compound: A Pass@k Crossover on a Free-Verifier Domain. arXiv:2606.07856 (2026)
11. Xu, Z., Li, Y., Jiang, F., Ramasubramanian, B., Niu, L., Lin, B., Poovendran, R.: TinyV: Reducing False Negatives in Verification Improves RL for LLM Reasoning. arXiv:2505.14625 (2025)
12. Yuan, Z., Yuan, H., Li, C., Dong, G., Lu, K., Tan, C., Zhou, C., Zhou, J.: Scaling Relationship on Learning Mathematical Reasoning with Large Language Models. arXiv:2308.01825 (2023)
13. Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Yue, Y., Song, S., Huang, G.: Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv:2504.13837 (2025)
14. Zelikman, E., Wu, Y., Mu, J., Goodman, N.D.: STaR: Bootstrapping Reasoning With Reasoning. arXiv:2203.14465 (2022)